# Refondre pour le cloud

Séance 2 – Développer pour le cloud – 2025 / 2026

Florian Jauffret – M2 Dev Full Stack – Maj : 03/2026

# Objectifs pédagogiques

- Identifier les problématiques potentielles d'une solution existante
- Procéder étape par étape pour procéder à la refonte de l'application
- Mettre en places les modifications nécessaires avant de rendre l'application « cloud ready », respectueuse des principes 12 facteurs.

# Plan

- Exemple de projet pédagogique
- Etapes de refonte
- Checklist de conformité cloud
- Outils à connaitre, voir maitriser
- Stratégie de production
- Exemple d’implémentation pas-à-pas

# Exemple de projet pédagogique

- Application simple « Todo List » : https://github.com/docker/getting-started-todo-app
- Application monolithique
- Base du futur projet de dev cloud
- Nécessite d'être refondue pour être compatible cloud natif
- Mise en place de l'authentification et du profil utilisateur conforme RGPD

# Etapes de refonte

Rendre l’application « cloud ready »

# Architecture et découpage

Constats probables :

- Le frontend et le backend sont insuffisamment découpé et découplés.
- La logique du métier est concentrée dans le backend avec peu de séparation.

Améliorations :

- Séparer clairement les responsabilités : frontend (SPA) et backend (API) comme deux services distincts, chacun avec son propre cycle de vie.
- Introduire des couches : routeur / contrôleur, service, repository pour faciliter les tests et l'évolution.
- Utiliser un fichier de configuration unique (ex: .env) pour les paramètres d'environnement plutôt que des variables dispersées dans le code.

# Gestion des données

Constats probables :

- Une base MySQL unique, avec accès direct depuis le backend.
- Pas de gestion de migrations versionnées.

Améliorations :

- Isoler la base de données comme service indépendant avec un schéma propre.
- Versionner les migrations (ex: avec knex ou typeorm) pour assurer une cohérence entre environnements.
- Utiliser un ORM/query builder pour éviter les injections SQL et faciliter les changements de base.

# Sécurité

Constats probables :

- Absence d'authentification (l'app est ouverte).
- Secrets (mots de passe base, clés) peut-être codés en dur ou dans le compose.yaml.

Améliorations :

- Ajouter une authentification (JWT, OAuth2) pour respecter les futures exigences RGPD.
- Ne jamais stocker de secrets en clair dans le code ou les fichiers de configuration. Utiliser des variables d'environnement et, en production, un gestionnaire de secrets (Vault, Azure Key Vault).
- Mettre en place des en-têtes de sécurité (CORS, CSP, Helmet.js).
- Valider et assainir toutes les entrées (frontend et backend).

# Configuration

Constats probables :

- Des valeurs d'environnement peuvent être partiellement définies dans le compose.yaml.
- Pas de distinction explicite entre environnements (dev, test, prod, etc.).

Améliorations :

- Externaliser complètement la configuration (variables d'environnement) et ne rien hardcoder.
- Utiliser des fichiers .env.example pour documenter les variables nécessaires.
- Séparer les environnements via des fichiers de configuration dédiés ou des valeurs par défaut surchargées.

# Observabilité

Constats probables :

- Les logs vont dans la console (Docker) sans structure.
- Pas de métriques ni de tracing.

Améliorations :

- Structurer les logs en JSON pour faciliter leur ingestion (par exemple avec pino).
- Ajouter des métriques (via prom-client) pour exposer les statistiques (requêtes, erreurs, latence, etc.).
- Instrumenter les appels (OpenTelemetry) pour permettre le tracing distribué ultérieurement.

# Tests

Constats probables :

- Peut-être quelques tests unitaires, mais pas de tests d'intégration ni de tests de charge.

Améliorations :

- Ajouter des tests unitaires pour les services et les contrôleurs.
- Mettre en place des tests d'intégration avec une base de données de test.
- Prévoir des tests de bout en bout pour le frontend.
- Automatiser les tests dans une pipeline CI.

# Dockerfile et image

Constats probables :

- Le Dockerfile actuel construit le frontend puis le sert avec le backend dans une seule étape.

Améliorations :

- Utiliser des builds multi-étapes pour optimiser la taille de l'image.
- Créer une image distincte pour le backend et une autre pour le frontend.
- Ne pas exécuter les processus en tant que root : créer un utilisateur non privilégié.
- Gérer les dépendances avec cache pour accélérer les builds.

# Documentation

Constats probables :

- Le README est minimal pour le démarrage mais manque de vision globale (procedure de lancement, developpement, ADR, etc.).

Améliorations :

- Documenter l'architecture (diagramme, choix techniques).
- Ajouter un guide de contribution.
- Détailler les variables d'environnement nécessaires en production.

# Bénéfices attendus

Ces améliorations sont préparatoires à la transformation cloud native. Elles permettent de :

- Faciliter le découpage en microservices (séparation front/back, couches métier).
- Respecter les 12 facteurs (configuration, logs, services attachés).
- Répondre aux exigences RGPD (authentification, traçabilité, sécurité des données).
- Poser les bases de l'observabilité et de la résilience.

# Checklist de conformité cloud

Basé sur les 12 facteurs et les bonnes pratiques

# Code base

- Un seul dépôt par microservice, ou monorepo bien structuré.
- Utilisation d'un système de version avec une branche main pour la production (ex : git)
- Étiquetage des versions pour chaque release (ex : tags).

# Dépendances

- Toutes les dépendances déclarées explicitement (ex : package.json).
- Aucune dépendance implicite (système, globale).
- Utilisation d'images de base minimales.
- Verrouillage des versions exactes (ex : package-lock.json).
- Isolation des dépendances de développement (ex : devDependencies).

# Configuration

- Externalisation complète de la configuration dans des variables d'environnement.
- Documentation des variables d'environnement nécessaires (ex : .env.example).
- Aucune configuration stockée dans le code (ex : secrets, URLs, API keys).
- Séparation des configurations par environnement via fichiers ou services externes.
- Utilisation d'un gestionnaire de secrets (Vault, Azure Key Vault) pour les données sensibles.

# Services de support

- Toutes les ressources externes traitées comme des ressources attachées (ex : base de données, cache, broker).
- Connexion via URL ou variables d'environnement, pas de configuration locale.
- Possibilité de remplacer un service de support par un autre sans changement de code (ex: MySQL → PostgreSQL).

# Build, release, run

- Séparation stricte des étapes :
  - build (création de l'image),
  - release (tag + déploiement),
  - run (exécution).
- Images Docker construites une seule fois et promues entre environnements.
- Utilisation d'un registre d'images pour stocker les images (ex : Azure Container Registry, Docker Hub).
- CI/CD automatisée pour construire et tester à chaque commit.

# Processus

- Services sans état (stateless) : aucune donnée persistante dans le conteneur.
- Sessions utilisateur externalisées (ex : Redis, base de données).
- Si état nécessaire, externalisé dans un service de support (ex : base de données).

# Port binding

- Chaque service expose son port via une variable d'environnement (ex : PORT).
- Les conteneurs écoutent sur 0.0.0.0 pour accepter les connexions externes.
- Les ports sont déclarés dans le Dockerfile (EXPOSE) et dans les orchestrateurs.

# Concurrency

- Conception permettant le scaling horizontal : pas de verrous globaux, pas de dépendance sur une instance particulière.
- Utilisation de processus légers pour tirer parti du multi-cœur (ex : clusters Node.js, workers).
- Configuration des variables de parallélisme via l'environnement (ex : WEB_CONCURRENCY).

# Disposability

- Démarrage rapide, moins de quelques secondes idéalement.
- Arrêt gracieux (graceful shutdown) : écoute des signaux SIGTERM, fin des traitements en cours, fermeture des connexions.
- Health checks implémentés pour permettre à l'orchestrateur de gérer les redémarrages (ex : liveness, readiness).

# Dev/prod parity

- Utilisation des mêmes services de support (versions identiques) en développement et production.
- Parité des environnements : Docker (ou Kubernetes) utilisé localement et en production.
- Réduction des écarts : pas de services “spéciaux” en développement (ex: pas de serveur de mail factice si production utilise un vrai service).

# Logs

- Logs émis sur la sortie standard (stdout) / erreur (stderr) au format structuré (JSON).
- Les logs ne contiennent pas de données sensibles.
- Centralisation des logs dans un système externe pour analyse (ex : Loki, Azure Log Analytics).

# Admin processes

- Les tâches d'administration (migrations, purges, imports) exécutées dans des conteneurs éphémères avec le même codebase et configuration.
- Utilisation de commandes spécifiques lancées via des jobs Kubernetes ou des scripts CI/CD (ex : npm run migrate).

# Sécurité

- Authentification et autorisation implémentées (ex : JWT, OAuth2).
- Chiffrement des données sensibles au repos et en transit (ex : TLS).
- Utilisation de mTLS entre services (ex : service mesh).
- Gestion des secrets, injection sans les exposer dans les variables d'environnement standards (ex : via Vault / Key Vault).
- Conteneurs non root (user nobody).
- Scan des vulnérabilités des images (ex : Trivy, Snyk).

# Observabilité

- Métriques exposées pour CPU, mémoire, latence, taux d'erreur (ex : Prometheus).
- Tracing distribué avec export vers Jaeger ou Azure Monitor (ex : OpenTelemetry).
- Dashboards pour visualiser métriques et logs (ex : Grafana).
- Alerting sur anomalies critiques (ex : Prometheus Alertmanager).

# Résilience

- Gestion des timeouts, retries avec backoff exponentiel, circuit breakers.
- Politiques de déploiement pour éviter les interruptions (ex : rolling update, blue/green).
- Tests de charge automatisés validant la capacité de scaling (ex : k6, Locust).

# RGPD

- Recueil du consentement explicite (interface utilisateur).
- Droit à l'oubli : suppression des données personnelles via un mécanisme asynchrone.
- Portabilité : API d'export au format JSON.
- Minimisation : ne stocker que les données strictement nécessaires.
- Journalisation des accès aux données personnelles.

# Outils à connaitre, voir maitriser

Les outils sur lesquels nous allons nous appuyer

# Services génériques : Conteneurisation & orchestration

- Docker : Création d'images, gestion multi-conteneur.
- Docker Compose : Environnement de développement local.
- Kubernetes : Orchestration des conteneurs (concepts : pods, services, deployments, ingress, configmaps, secrets, HPA).
- kubectl : Ligne de commande pour interagir avec Kubernetes.
- Helm : Gestionnaire de packages Kubernetes (templates, releases).
- KEDA : Autoscaling événementiel (basé sur Kafka, etc.).

# Services génériques : Développement & langages

- Langage backend (ex : Node.js) – Bonnes pratiques (asynchrone, packages, structure modulaire).
- Framework frontend (ex : React) – SPA, routage, gestion d'état.
- Outil d'analyse de code (ex : SonarQube) – Analyse de qualité de code.
- TypeScript (optionnel mais recommandé) – Typage statique.

# Services génériques : Communication asynchrone

- Apache Kafka : Broker d'événements (topics, producteurs, consommateurs, consommateurs groupés).
- Schema Registry : Gestion des schémas Avro/Protobuf pour garantir l'interopérabilité (facultatif).
- Kafka CLI / Confluent CLI : Administration de base.

# Services génériques :
# Bases de données & cache

- PostgreSQL – Base de données relationnelle (migrations, index, isolation).
- Redis – Cache et stockage de sessions.
- pgAdmin / DBeaver – Interfaces d'administration (optionnel).

# Services génériques : Authentification & sécurité

- Keycloak : IAM open source (OIDC, OAuth2, gestion des utilisateurs, consentement RGPD).
- HashiCorp Vault : Gestion de secrets, injection dans les pods.
- Istio / Linkerd : Service mesh (mTLS, observabilité, politiques).
- Open Policy Agent (OPA) : Contrôle d'accès fin (optionnel).

# Services génériques : Observabilité

- Prometheus : Collecte de métriques (pull).
- Grafana : Visualisation (dashboards, alertes).
- Loki : Agrégation de logs (push via Promtail).
- Jaeger : Tracing distribué (compatible OpenTelemetry).
- OpenTelemetry : Instrumentation des applications (traces, métriques, logs).

# Services génériques : CI/CD & IaC

- GitHub Actions (ou GitLab CI) : Pipelines de build, test, push d'images.
- ArgoCD : Déploiement continu GitOps (synchronisation avec Kubernetes).
- Terraform : Infrastructure as Code multi-cloud (pour la création du cluster AKS, du VNet, etc.).
- Docker Desktop K8s : Clusters Kubernetes locaux pour le développement.

# Services génériques : Tests

- Framework de test (ex : Jest, Vitest) – Tests back-end.
- Framework de test UI (ex : Playwright) – Tests end-to-end.
- Framework spécialisés (ex : k6) – Tests de charge.
- Framework de conteneurs (ex : Testcontainers) – Tests d'intégration avec des services conteneurisés.

# Services génériques : Sécurité des images

- Trivy – Scan des vulnérabilités dans les images Docker.

# Stratégie de production

Passer du local à un environnement cloud

# Environnements

- Local :
  Développement sur poste de travail avec Docker Compose (ou kind + tilt) pour itération rapide.
- Staging :
  Environnement préproduction sur Azure (AKS) identique à la production, avec données anonymisées.
- Production :
  Cluster AKS avec haute disponibilité, scaling automatique, sauvegardes.

# Gestion de la configuration

- Utilisation de Helm charts pour paramétrer les déploiements selon l'environnement.
- Variables d'environnement injectées via ConfigMaps et Secrets (Azure Key Vault + CSI driver).
- Fichiers de valeurs Helm séparés : values-dev.yaml, values-staging.yaml, values-prod.yaml.

# Pipelines de CI/CD

CI (à chaque push sur branches principales) :

- Linting, tests unitaires, tests d'intégration.
- Build des images Docker (multi-architecture).
- Scan de sécurité (Trivy).
- Push des images vers Azure Container Registry (ACR) avec tag git-sha et latest (pour staging).

CD (déploiement automatisé) :

- Staging : déploiement automatique sur AKS staging via Helm upgrade.
- Production : déclenché manuellement (ou via tag) après validation.
- Stratégie blue/green ou rolling update avec évaluation des health checks.

# Gestion des secrets en production

- Stockage : Azure Key Vault.
- Injection : CSI driver pour monter les secrets en tant que volumes dans les pods.
- Pas de stockage de secrets dans les variables d'environnement des pods (évite l'exposition via kubectl describe).

# Observabilité

- Déploiement d'une stack Prometheus + Grafana dans le cluster (ou utilisation d'Azure Monitor pour les métriques).
- Logs centralisés : Azure Log Analytics ou Loki avec Grafana.
- Tracing : Jaeger (ou Azure Application Insights) avec instrumentation OpenTelemetry.

# Sauvegarde et récupération

- Sauvegarde quotidienne des bases de données
  (ex : Azure Database for PostgreSQL flexible server avec sauvegarde automatique).
- Scripts de restauration testés périodiquement.

# Documentation et runbooks

- Guide de déploiement (README avec commandes Helm).
- Diagrammes d’architecture (C4 model).
- Runbooks pour les incidents (redémarrage, restauration, scaling).

# Validation locale avant production

- Utilisation de Docker Compose avec la même stack que la cible (Kafka, PostgreSQL, Redis, Keycloak) pour valider localement.
- Tests end-to-end (Playwright) exécutés contre l'environnement de test local.

# Exemple d’implémentation pas-à-pas

Comment procéder concrètement

# Exemple d’implémentation

- Refactorisation du code monolithique en microservices (cf. checklists).
- Création des images Docker optimisées (multi-stage, non-root).
- Configuration de Docker Compose pour l’environnement de développement local (incluant Kafka, PostgreSQL, Keycloak).
- Mise en place des pipelines CI/CD (GitHub Actions) construisant les images et les poussant vers ACR.
- Création du cluster AKS avec Terraform (ou Bicep) : réseau, identités managées, intégration ACR.
- Déploiement des Helm charts pour chaque microservice.
- Configuration de l’observabilité (Prometheus, Grafana, Loki) dans le cluster.
- Mise en place des politiques réseau (Calico) et du service mesh (Istio) pour la sécurité.
- Tests de charge et validation des auto-scaling (KEDA basé sur Kafka).
- Documentation et transfert de connaissances.